

MELANOSCOPE: Melanoma Detection System

MINOR PROJECT REPORT

Submitted in partial fulfillment of the requirement for the degree

of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

by

Manya Bora

Aditya Gupta

Avinash Thakur

Enroll No.: 35496302721

Enroll No.: 00396302721

Enroll No.: 02796302721

Guided By

Dr. Neeti Sangwan
Asst. Professor



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY
(AFFILIATED TO GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY,
DELHI)
DELHI- 110058

November 2024

CANDIDATE'S DECLARATION

It is hereby certified that the work which is being presented in the B. Tech Minor Project Report entitled "**MELANOSCOPE: Melanoma detection system**" in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** and submitted in the **Department of Computer Science & Engineering** of **MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY, New Delhi (Affiliated to Guru Gobind Singh Indraprastha University, Delhi)** is an authentic record of our own work carried out during a period from **August 2024 to December 2024** under the guidance of **Dr. Neeti Sangwan, Asst. professor.**

The matter presented in the B. Tech. Minor Project Report has not been submitted by me for the award of any other degree of this or any other Institute.

Manya Bora

Aditya Gupta

Avinash Thakur

(En. No.: 35496302721)

(En. No.: 00396302721)

(En. No.:02796302721)

CERTIFICATE

This is to certify that the minor project entitled "**MELANOSCOPE: Melanoma detection system**" made by the candidates is correct to the best of my knowledge. They are permitted to appear in the External Minor Project Examination.

Dr Neeti Sangwan
Asst. Prof.

Dr. Nishtha Jatana
HoD, CSE

ACKNOWLEDGEMENT

This acknowledgement is dedicated to all the individuals who provided us with assistance and support in creating this project. The project could not have gone smoothly without the help of these people.

We express our deep gratitude to **Dr. Neeti Sangwan**, Asst. Professor, Department of Computer Science & Engineering for her valuable guidance and suggestion throughout our project work.

We would like to extend our sincere thanks to **Head of the Department, Computer Science, Dr. Nishtha Jatana** for her time-to-time suggestions to complete our project work. I am also thankful to **Prof (Dr.) Archana Balyan, Director, MSIT** for providing us the facilities to carry out our project work.

We give our heartfelt thanks to the faculty for their guidance and constant supervision. Secondly, we would also like to thank our parents and friends who provided us with motivation and support in finalizing this project within the limited time frame.

Manya Bora

Aditya Gupta

Avinash Thakur

(En. No: 35496302721)

(En. No: 00396302721)

(En. No.:02796302721)

TABLE OF CONTENTS

TITLE PAGE	i
CANDIDATE’S DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGMENT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
ABSTRACT	ix
Chapter 1: Introduction	1-4
1.1 Introduction	1
1.2 Motivation	2
1.3 Objectives	2
1.4 Summary of the Report	3
Chapter 2: Literature review	6-9
Chapter 3: Methodology and Techniques	10-28
3.1 Technology and libraries used	10
3.1.1 Python	10
3.1.2 Numpy	10
3.1.3 Tensor Flow	11
3.1.4 Keras	12
3.1.5 Pandas	12
3.1.6 Seaborn	13
3.2 Data Collection	14
3.3 Exploratory Data Analysis	14
3.4 Data Preprocessing	15

3.5 Convolutional neural networks	16
3.5.1 Multilayered structure of CNN	17
3.5.2 Regularization	18
3.5.3 Applications of CNN	20
3.5.4 Advantages of CNN	20
3.6 Model Architecture	21
3.6.1 Summary of model A	22
3.6.2 Summary of model B	23
3.6.3 The Ensemble model	24
3.6.4 Comparison between model A and model B	26
Chapter 4: Results and discussions	29-35
4.1 Model Performance metrics	29
4.2 Model A Evaluation	29
4.2.1 Training results	30
4.2.2 Testing results	30
4.2.3 Benefits and drawbacks	30
4.3 Model B Evaluation	31
4.3.1 Training results	31
4.3.2 Testing results	31
4.3.3 Benefits and drawbacks	31
4.4 Final Result	32
4.5 Future Scope	34
Chapter 5: Conclusion	36
References	37

LIST OF FIGURES

Figure 1.1	Model development flow	4
Figure 3.1	Logo of Python	10
Figure 3.2	Logo of Numpy	11
Figure 3.3	Logo of TensorFlow	11
Figure 3.4	Logo of Keras	12
Figure 3.5	Logo of Pandas	13
Figure 3.6	Logo of Seaborn	13
Figure 3.7	A few images from the HAM10000 dataset	14
Figure 3.8	Correlation heatmap: age vs gender	15
Figure 3.9	Model A training and test epoch a) loss and b) accuracy respectively.	21
Figure 3.10	Model B training and test epoch a) loss and b) accuracy respectively.	21
Figure 3.11	Summary of model A	22
Figure 3.12	Summary of model B	24
Figure 3.13	Code snippet of the ensemble model	25
Figure 3.14	Architecture of Ensemble model	26
Figure 4.1	Test Accuracy of Model A	32
Figure 4.2	Test Accuracy of Model B	32
Figure 4.3	Test Accuracy of ensemble model	33
Figure 4.4	Confusion matrix of the ensemble model	33
Figure 4.5	Correlation matrix	33

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
CCE	Categorical Cross Entropy
MSE	Mean Squared Error
MNIST	Modified National Institute of Standards
ISIC	International Skin Imaging Collaboration
HAM	Humans Against Machine
DL	Deep Learning

ABSTRACT

This project aims to build a melanoma detection system by employing the concepts of deep learning, a branch of artificial intelligence. Melanoma is a type of skin cancer that is caused in melanocytes. Melanocytes are the skin cells which produce melanin in the skin. Early and accurate diagnosis of melanoma is required before the cancer cells spread to other non-tumor sites for higher survival rates since melanoma is highly lethal. The HAM1000, Humans Against Machines 10000, dataset collected in the ISIC challenge which has a collection of over 10,000 dermoscopic images is used to train and test the data. In our research, Convolutional Neural Network models are trained and tested through various python libraries like TensorFlow, Keras etc., by using image segmentation. Two CNN architectural models are built and then ensembled together for enhanced performance accuracy.

CHAPTER 1: INTRODUCTION

1.1 Introduction

The skin is the largest organ of the human body. About 15% of a human's body weight is contributed by their skin. Since, a large part of our lives involves being exposed to the sun, skin has a high exposure to the sunlight. Sunlight provides vitamin D which is essential for our body and therefore, cannot be avoided. This poses a huge problem as it also provides a gateway to ultraviolet ray exposure. Although exposure to sunlight is beneficial to us, it is also one of the leading causes of melanoma and is found to accumulate the melanoma-causing factors over time. Therefore, we must take precautions and ensure that any symptoms of this potentially life-threatening disease are found as early as possible. Identification of the melanoma signs in the initial stages provide higher chances of successful diagnosis.

Melanoma is a type of skin cancer. It begins in the pigment producing skins cells of the body known as melanocytes. Melanocytes produce melanin which is what provides colour to the skin. The lesions appear in irregularly shaped or unusually coloured marks or moles on the skin. It is highly invasive and should be diagnosed in the early stages which allow successful treatment through surgery alone. Once the cancer cells start to spread to parts of the body i.e. once metastasis begins, survival rates drop significantly. For melanoma detection, a visual skin exam or biopsy, a process of examining lesions by surgically removing them, is conducted in which suspicious-looking pigmented areas and existing moles are examined for abnormality and signs of cancer. Since this process is very time consuming and tedious, many disregard the seriousness of skin conditions and delay the doctor's appointment. About 30% melanomas start in existing moles which are often free of suspicion. Therefore, an automated way to identify melanoma through pictures is proposed which will encourage people to take skin conditions seriously and alerts people when it is time to give serious attention for it.

This project employs the usage of concepts of deep learning and convolutional neural networks for building a predictive model for melanoma detection. Deep learning is a subset of machine learning and mimics the functioning of the human brain by making multilayered networks made of neurons. These layers are interconnected and pass their

output to the next as input until the end to refine and optimize the prediction or classification. CNNs are used in computer vision and image classification related tasks.

1.2 Objectives

- **Data collection**

Gather an extensive dataset of melanoma and non-melanoma dermoscopic images. Ensure that the collection must be diverse encompassing variations in skin types and demographics so as to reduce any bias enhancing the model's generalizability.

- **Data Analysis**

Perform thorough analysis using visual representations via python libraries to uncover the relationships between data and related factors like age, gender, and localization.

- **Data Pre-processing**

Cleaning and transforming raw data into a suitable format using various normalization and data augmentation techniques like image resizing, rotation and shifting.

- **Model Development**

Design and implement CNN architecture to create a deep learning model. Define each layer according to its function.

- **Prediction**

Detect melanoma and non-melanoma images accurately using the trained CNN predictions. Evaluate model using performance metrics like accuracy, precision, F1-score etc.

1.3 Motivation

Melanoma, one of the deadliest forms of skin cancer, has a significantly higher survival rate when detected early. However, traditional diagnostic methods rely heavily on visual inspection by dermatologists, which can be subjective and time-consuming. This underscores the urgent need for an automated, accurate, and scalable solution to aid in melanoma detection. By leveraging advancements in deep learning and Convolutional

Neural Networks (CNNs), this project aims to provide an efficient diagnostic tool that reduces the burden on healthcare professionals while increasing early detection rates.

Deep learning models, especially CNNs, have demonstrated exceptional performance in image-based tasks such as classification and segmentation. Applying this technology to dermatoscopic images presents a promising opportunity to enhance diagnostic accuracy. The integration of this tool into clinical workflows has the potential to save lives by identifying melanoma cases that might otherwise go unnoticed.

1.4 Summary of the Report

This project focuses on the application of Convolutional Neural Networks (CNNs) to detect melanoma, a critical form of skin cancer, from dermatoscopic images. By leveraging advanced deep learning techniques, the aim is to provide an efficient, accurate, and scalable solution to support healthcare professionals in early melanoma diagnosis. The project integrates data collection, preprocessing, and analysis with robust CNN architectures—Model A, Model B, and an ensemble model. These models were developed and evaluated to achieve high accuracy and reliability in classifying melanoma versus non-melanoma cases.

The report is structured systematically to cover the research process, starting with an introduction to the problem and concluding with the results and implications. Each chapter provides detailed insights into the project workflow, offering a comprehensive understanding of the methodologies and techniques employed. The introduction highlights the project's motivation and objectives, including data collection, preprocessing, and model development, setting the stage for the work. A review of existing research follows, discussing previous advancements in melanoma detection, CNN-based methodologies, and gaps in current approaches, thereby justifying the need for this project. The methodology section dives into the tools, libraries, and techniques used, including data analysis and preprocessing strategies, and explains the structure and functionality of CNNs. Detailed descriptions of the architectures of Model A, Model B, and the ensemble model are included, showcasing their design and performance optimization methods. The results and discussion section evaluates the models using metrics like accuracy, precision, recall, and F1 score, comparing their

performance during training and testing. Strengths and limitations of each model are addressed, and the advantages of the ensemble approach are highlighted. Finally, the report concludes by summarizing the project outcomes, emphasizing its potential impact in healthcare, and suggesting directions for future work. References at the end ensure credibility and provide insight into the foundational materials that informed the project.

Fig 1.1, demonstrates the process of model development implemented to create the system.

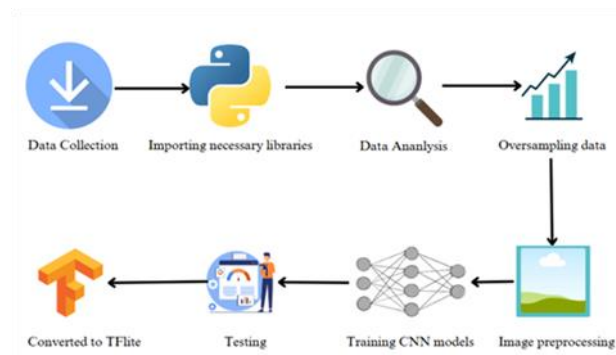


Fig 1.1: Model development flow

- The HAM10000 dataset, which is a source of over 10000 dermoscopic images, is collected from Harvard dataverse.
- Libraries like numpy, pandas, matplotlib, RandomOverSampler etc, are imported.
- The dataset is analysed to understand underlying relationships between data and different factors like localization, age and gender distribution.
- Oversampling of data is performed which increases the data samples from 10005 images to 46935 images.
- Image preprocessing techniques which include rotation, zooming, rescaling and normalization of pixels are applied.
- Data is split between training and testing counterparts in the ratio of 80 to 20. Two CNN model architectures are defined tailored to custom activation and loss function.

- The two models are tested and converted to Tflite format to then be ensembled.
- The two models are aggregated using average function to form an ensemble model. This is converted to a TFLite format for use.

CHAPTER 2: LITERATURE REVIEW

Adekanmi A. Adegun et al [6] proposed a deep-learning method for pixel-wise classification using softmax classifier. The system employed an enhanced encoder-decoder network connected through skip pathways. They achieved an accuracy and dice coefficient of 95% and 92% on ISIC dataset.

Chandran Kaushik Viknesh et al [7] introduced a computer-aided detection technique for early melanoma diagnosis. They proposed two methods to investigate the relationship between model depth and performance with varying dataset sizes. The first method employs convolutional neural networks, AlexNet, LeNet, and VGG-16, while the second uses support vector machines with a default RBF kernel to arrive at an accuracy rate of 91% and 86.6% respectively.

In [8], Devika Moturi, Ravi Kishan Surapaneni and Ventaka Sai Geethika Avanigadda reiterate the importance of early detection and precise diagnosis for determining cancer therapies. The suggested technique utilizes deep learning methods like MobileNetv2 and Dense net for classifying benign and malignant tumors.

Balazs Harangi [9] investigates creation of an ensemble of deep convolutional neural networks to combine the strengths of different models and further improve the accuracy in identification of three classes of skin cancer. This approach allows to achieve great results even in the absence of a large dataset of labelled images. The outputs of four different deep neural networks, GoogleNet, AlexNet, ResNet, and VGGNet, are fused together to get average area under the receiver operating characteristic curve to be 0.891 on a dataset of the IEEE International Symposium on Biomedical Imaging (ISBI) 2017 challenge on Skin Lesion Analysis Towards Melanoma Detection.

Manju C.P., Jeslin P. JO and Vinitha V. [10] discuss squamous, basal cell and melanoma as the three primary subtypes of skin cancer. They trained the model with HAM10000 dataset, a lesion-based image dataset. Enforcing an autoencoder for feature extraction followed by classification with the help of v3 pretrained model and 3D CNN

leading to an accuracy of 0.96. The help of an autoencoder as a feature extraction brings enhances efficiency and performance.

Umesh Kumar Lilhore et al [11] aim to increase the precision and efficacy of skin cancer recognition and enhance patients' experiences. They integrate Standard U-Net and Improved MobileNet-V3 with optimization techniques in the deep learning training process which makes system to differentiate between malignant and benign lesions. Adjustments in the model architecture are made like using the crossed-entropy loss function.

In [12], Yuexiang Li and Linlin Shin proposed two deep learning methods that focus on the three necessary tasks of image processing of skin lesions namely lesion segmentation, lesion dermoscopic feature extraction and lesion classification. The deep learning framework was experimented on the ISIC 2017 dataset by using a CNN model for feature extraction and two fully convolutional residual networks (FCRN) for segmentation result and classification. Accuracies of 0.753, 0.848 and 0.912 were received for the tasks respectively.

Jinen Daghrir et al [13] proposed a system of combination of three different methods- a convolutional neural network and two classical machine learning classifiers trained with a set of features describing the texture, color and borders of the skin lesions. CNN along with KNN and SVM methods were each tested. Result aggregation of the different methods enhances the overall performance to an accuracy of 88.4%.

In [14], Shubhendu Banerjee et al present a deep learning based 'You Only Look Once (YOLO)' algorithm which works on the basis of the application of deep convolutional neural networks to detect melanoma from dermoscopic and digital images. They aimed to offer a more precise alternative to conventional CNNs by incorporating a two-phase process combining graph theory and fuzzy number-based approximation heightened segmentation results. It yielded a Jac score of 79.84% on the ISIC 2019 dataset.

Ghadha Alwakid et al [15] put forward an approach using deep learning as a method for extracting a lesion zone with precision. To improve the image quality, Enhanced Super-Resolution Generative Adversarial Networks (ESRGAN) is used, followed by segmentation to segment Regions of interest (ROI) from the full image. The image is

evaluated with a CNN and a modified version of Resnet-50 to classify skin lesions. The CNN-based model gave an accuracy and precision of 0.86 and 0.84 respectively on the ISIC 2018 dataset.

Uzma Bano Ansari and Tanuja Sarode [16] use digital image processing techniques along with Support Vector Machine for skin cancer detection. It provides a step up from the biopsy method of skin cancer detection by eliminating the need for unnecessary excision of moles. Gray level co-occurrence matrix (GLCM) is used for feature extraction and captures features like contrast, mean, energy and homogeneity. SVM is then used for regression analysis and classification of cancerous and non-cancerous images.

Ioannis Kousis et al [17] present training and testing of 11 convolutional neural network architectures using the HAM10000 dataset which include a variety of seven skin lesion classes. The data is fine tuning and augmented to solve the imbalance problem for training. Through testing, DenseNet169 is found to produce the best results with an accuracy of 92.25%, a recall of 93.59% and an F1-score of 93.27% which outperforms existing state-of-the-art efforts. An android mobile application was also constructed using the light version of DenseNet169 to classify benign or malignant lesion types along with providing the user with information about the allowed sun exposition time based on the current UV radiation degree etcetera.

A Dascalu and EO David [18] utilized a convolutional neural network architecture based on the Inception V2 network to determine the impact of image quality on the accuracy of diagnosis by sonification employing a rudimentary skin magnifier with polarized light (SMP). A k-means clustering algorithm was employed to cluster the activations SMP achieved a F2 score sensitivity of 91.7%.

MF Jojoa Acosta et al [19] propose a method where the region of interest within a dermatoscopic image is automatically cropped using the Mask and Region-based CNN technique, followed by classification of lesion images as benign or malignant using a ResNet152 structure. It was evaluated on the ISIC 2017 challenge and suggests a better performance over the previously proposed methods. The eVida M6 model provides an overall accuracy of 0.904, sensitivity of 0.820 and specificity of 0.925.

In [20], Marriam Nawaz et al present a fully automated method for segmenting the skin melanoma at its earliest stage by employing a deep-learning-based approach. Region-based Convolutional Neural Network (RCNN) is used along with fuzzy k-means clustering (FKM). The dataset images are first pre-processed before applying the RCNN to obtain the feature vector of fixed length. The melanoma-affected portion is then segmented using FKM. The proposed method attained an average accuracy of 95.49, 933.1 and 95.6% on the ISIC-2016, ISIC-2017, and PH2 datasets, respectively.

CHAPTER 3: METHODOLOGY AND TECHNIQUES

3.1 Technologies and libraries

This project was done on Google Colab which is a hosted Jupyter Notebook service that runs entirely in the cloud. Following are the various technologies and important libraries used in the project work:

3.1.1 Python

Python, fig. 3.1, is a popular and easy-to-learn programming language widely used in fields like web development, data science, and automation. It uses simple, straightforward syntax that resembles English, making it ideal for beginners and professionals alike. Python has a huge library of pre-built modules, or "libraries," which save time and make it possible to quickly create everything from websites to complex data analysis projects. Additionally, Python is highly flexible, and can be used for many different types of projects, from small scripts to large, data-heavy applications etc.



Fig. 3.1: Logo of Python

3.1.2 Numpy

NumPy (Numerical Python), fig. 3.2, is a foundational library for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices. It also includes many mathematical functions to perform operations on these arrays efficiently. By enabling quick and easy array manipulations, NumPy is essential for data science

and machine learning tasks, allowing users to perform complex calculations and statistical operations with simple code.



Fig. 3.2: Logo of NumPy

3.1.3 TensorFlow

TensorFlow, fig. 3.3, is a powerful, open-source machine learning framework developed by Google that allows developers to create and train machine learning models, especially neural networks. Its flexibility and performance make it ideal for large-scale machine learning and deep learning projects. TensorFlow also supports multiple languages, but it's most commonly used with Python. With an extensive set of tools and support for both CPUs and GPUs, TensorFlow has become a core tool for AI research and development.



Fig. 3.3: Logo of TensorFlow

3.1.4 Keras

Keras, fig. 3.4, is an easy-to-use, high-level neural network library that runs on top of TensorFlow. Designed to make deep learning accessible, Keras offers a simple and user-friendly API, allowing developers to quickly build and test neural networks. Keras handles most of the complex computations in the background, so users can focus on building and experimenting with their models. It's often used for prototyping and supports popular neural network layers, including convolutional and recurrent networks.



Fig. 3.4 Logo of Keras

3.1.5 Pandas

Pandas, fig. 3.5, is a popular library used for data manipulation and analysis in Python. It provides tools for handling data structures like Data Frames, which make it easy to manipulate, filter, and transform data. Pandas is particularly helpful when working with tabular data, such as data from spreadsheets or databases, making it a go-to tool for data wrangling in data science and analytics projects.



Fig. 3.5: Logo of Pandas

3.1.6 Seaborn

Seaborn, fig 3.6, is a Python library built on top of Matplotlib, designed to make it easy to create informative and attractive statistical graphics. With Seaborn, you can quickly visualize data distributions, trends, and patterns, making it easier to understand your data. Seaborn's functions are intuitive and allow users to create complex visualizations, like heatmaps and violin plots, with minimal code. It's a favorite among data scientists for exploring and presenting data insights.



Fig. 3.6: Logo of seaborn

3.2 Data Collection

The HAM10000, i.e. “Human Against Machine with 10000 training images”, contains a collection of a whopping 10,015 dermatoscopic images belonging to seven different classes namely melanocytic nevi, melanoma, benign keratosis-like lesions, basal cell carcinoma, actinic keratoses, vascular lesions, and dermatofibroma. It contains a broad range of melanocytic and non-melanocytic lesions and therefore, tackles the previous limitations of small-sized and homogeneous images by introducing diversity. It was collected for academic research for an ISIC (International Skin Imaging Collaboration Laboratory) challenge. A few images of the dataset are shown in fig. 3.7.

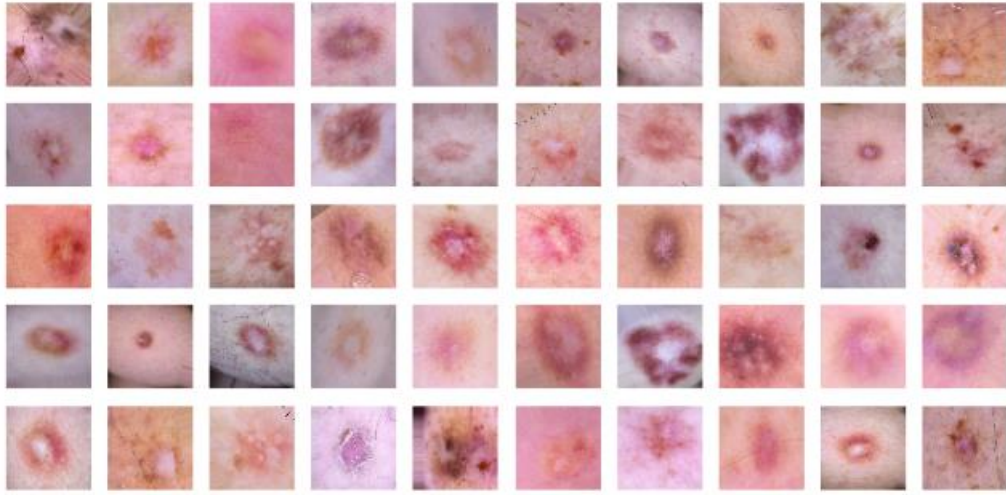


Fig. 3.7: A few images from the HAM10000 dataset

3.3 Exploratory Data Analysis

To work with data, we need to understand it and its underlying relationships between the factors. The meta data is read from the csv file and using python libraries like numpy, pandas and seaborn, data is represented in a graphical format. Histograms, bar plots, box plots, correlation heatmaps, violin plot etc. are used for data visualization and thus, enhancing comprehension. Various classes of skin cancer along with factors like frequency, age distribution, gender, localization, diagnoses are studied. Given fig 3.8, provides an example of a correlation heatmap of age vs gender. It is found that there is no significant correlation between age and gender as -0.17 is very close to 0.

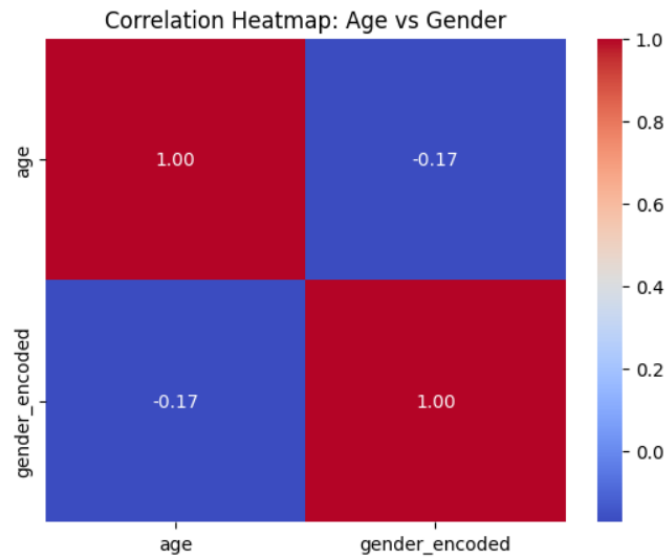


Fig. 3.8: Correlation heatmap of age vs gender

3.4 Data Preprocessing

The foundation of a machine learning model is a fine-tuned dataset with minimized errors so, we must clean the dataset for any irregularities or bias occurring. By observing its shape, it can be inferred that the dataset is skewed towards a particular class. This might make the machine learning model biased towards the majority class and hence, would result in making faulty predictions that favour one more than the others. This is prevented by introducing oversampling to the other classes and reaching a balance to acquire a bias-free collection. This is done by increasing the class samples by randomly duplicating certain examples from the minority class and adding them to it. The data is then reshaped to a 4D vector, (46935, 28, 28, 3), to better represent the image dataset by defining the number of samples now increased from 10,000 to 46,935, pixel size resized to 28x28 and the number of channels for RGB value-3 respectively.

To make the dataset more suitable for training, image augmentation techniques are applied. The pixel size of each image is reduced along with application of random rotations by ten degrees and random zoom ins and outs by 10%. Width and height are also shifted by 10% randomly. This makes the model more robust by creating different perspective views of the images.

3.5 Convolutional Neural Networks

Convolutional Neural Networks, abbreviated as CNNs, are a class of deep learning networks primarily designed for tasks related to computer vision, object identification and image classification. They imitate the hierarchical structure and functioning of the components of the human brain. They consist of a network of interconnected layers that each perform a specific role. They are distinguished by their ability to automatically and adaptively learn spatial hierarchies of features from input images. Unlike traditional machine learning techniques, CNNs do not require manual feature extraction; instead, they learn feature representations directly from raw data. This property makes CNNs particularly well-suited for image processing tasks.

Forward and backward propagation are essential processes in training Convolutional Neural Networks. Forward propagation involves passing an input, such as an image, through the network to generate an output prediction. This process starts with the input layer, where the raw data is fed into the network. In the convolutional layers, filters slide over the input data, performing convolution operations to produce feature maps. Non-linear activation functions like ReLU are applied to these feature maps to introduce non-linearity. The pooling layers follow, reducing the spatial dimensions of the feature maps through techniques like max pooling or average pooling, which retain the most significant features. Finally, the processed data flows into the fully connected layers, where the flattened feature maps are used to perform high-level reasoning, ultimately leading to the output layer. The output layer produces predictions, such as class probabilities in classification tasks, using activation functions like softmax.

Backward propagation, on the other hand, is the process of refining the network's weights and biases by minimizing the error between the predicted and actual outputs. This begins with the computation of loss using a suitable loss function, such as categorical cross-entropy for classification problems. Gradients of the loss with respect to the model's parameters are then computed using the chain rule of calculus. These gradients are propagated backward through the network, starting from the output layer and moving towards the earlier layers. Each layer computes the gradient of its parameters based on the error signal received from the subsequent layer. The parameters are updated using optimization algorithms like stochastic gradient descent

or Adam. This update process ensures that the model learns to improve its predictions over successive iterations.

Forward propagation is primarily concerned with generating predictions by applying learned weights, while backward propagation focuses on correcting errors by adjusting these weights and biases. Together, they form the backbone of CNN training, ensuring that the network not only maps input features to outputs effectively but also generalizes well to unseen data. These iterative processes continue until the loss function converges, signifying that the model has reached an optimal state of learning.

3.5.1 Multilayered structure of CNN

A convolutional Neural Network has a multilayered architecture designed to capture complex patterns in the image data. It typically stacks convolutional, activation and pooling layers that allows it to learn complex features. The different layers are described as follows:

a. Convolutional layers

It is designed to extract meaningful features from an input image through the application of small filters known as kernels. Convolution, a mathematical task, applies a sliding window function to a matrix of image pixels. Each filter detects specific patterns within the image and calculates the dot product between kernel weight and the corresponding input image. The result, which is a feature map, is passed as input to the next layer for refining and optimization. These feature maps capture specific patterns such as edges, textures, or other visual cues, depending on the layer's depth.

b. Activation Layer

It introduces non-linearity to the model by application of a non-linear function to the output gathered from the convolutional layers. This allows the model to capture complex relationships that cannot be handled by linear functions. Activation functions transform the output of neurons in a layer before passing it to the next layer. Rectified Linear Unit, ReLU, is the most common activation function and performs a transformation after every convolution replacing negative values with a zero.

c. Pooling Layer

This layer reduces the size of volume enhancing the computation process and decreasing chances of overfitting. This is done using the application of aggregation functions which reduce the spatial dimensions of the feature maps and provide focus to the essential information. Max pooling is the most common type of pooling. In this, the maximum value of a particular part is selected. Some other pooling methods are sum pooling and average pooling.

d. Flattening

The resulting feature maps are transformed into a one-dimensional vector after the convolution and pooling layers which is then passed as output to the fully connected layers.

e. Fully connected layers

This is the final layer of the CNN architecture and is responsible for performing classification or regression-based tasks on the extracted features. These layers are known as fully connected because each neuron in one layer is connected to every neuron in the preceding layer. This helps in creating a highly interconnected network.

f. Output Layer

This layer provides the final classification or prediction. The previous inputs are condensed into a final output vector. This vector contains the probabilities for each class. Logistic functions like softmax and sigmoid are used for this in case of classification tasks.

3.5.2 Regularization

Regularization is a technique used in machine learning and deep learning to prevent overfitting and improve the generalization performance of a model. It involves adding a penalty term to the loss function during training. It helps to prevent overfitting.

This penalty discourages the model from becoming too complex or having large parameter values, which helps in controlling the model's ability to fit noise in the training data. Regularization in deep learning methods includes L1 and L2 regularization, dropout, early stopping, and more. By applying regularization for deep learning, models become more robust and better at making accurate predictions on unseen data.

Applying the regularization technique may slightly increase the bias but slightly reduces the variance.

a. L1 Regression

It is also known as Lasso Regression. The general cost function is updated by the addition of a small bias known as the regularization term to receive better long-term predictions with its formula, eq. 1. The penalty term contains the product of the absolute weights with lambda. It can shrink the slope to 0 due to usage of absolute values.

$$L(x, y) = \min \left(\sum_{i=1}^n (y_i - w_i x_i)^2 + \lambda \sum_{i=1}^n |w_i| \right) \quad (1)$$

b. L2 Regression

It is known as Ridge Regression. A small amount of bias is introduced to reduce the complexity of the model. The amount of bias added to the model is known as Ridge Regression penalty. We can compute this penalty term by multiplying with the lambda to the squared weight of each individual features with eq. 2 as formula.

$$L(x, y) = \min \left(\sum_{i=1}^n (y_i - w_i x_i)^2 + \lambda \sum_{i=1}^n (w_i)^2 \right) \quad (2)$$

c. Dropout

In machine learning, “dropout” refers to the practice of disregarding certain nodes in a layer at random during training. A dropout regularization in deep learning is a regularization approach that prevents overfitting by ensuring that no units are codependent with one another.

d. Early Stopping

In this technique, the training is paused before the model starts learning the noise within the model. In this process, training is performed iteratively while measuring the performance of the model after each iteration until the new iterations are. After a point, the performance on the validation set gets worse and the model begins to overfit the training data; hence process is immediately stopped before the learner passes that point. Stopping the training process before the model starts capturing noise from the data is known as early stopping.

3.5.3 Applications of CNN

- Image classification: They are the state-of-the-art models for image classification. They can be used to classify images into different categories, such as cats and dogs, cars and trucks, and flowers and animals.
- Object detection: They can be used to detect objects in images, such as people, cars, and buildings. They can also be used to localize objects in images, which means that they can identify the location of an object in an image.
- Image segmentation: They can be used to segment images, which means that they can identify and label different objects in an image. This is useful for applications such as medical imaging and robotics.
- Video analysis: They can be used to analyse videos, such as tracking objects in a video or detecting events in a video. This is useful for applications such as video surveillance and traffic monitoring.

3.5.4 Advantages of CNN

There are several advantages of CNN's:

- They can achieve state-of-the-art accuracy on a variety of image recognition tasks, such as image classification, object detection, and image segmentation.
- They can be very efficient, especially when implemented on specialized hardware such as GPUs.

- CNNs are relatively robust to noise and variations in the input data.
- CNNs can be adapted to a variety of different tasks by simply changing the architecture of the network.

3.6 Model Architecture

Two machine learning models are set up using the concepts of Convolutional Neural Networks and Deep Learning.

The two models are trained and tested separately before creating the ensemble of the combination of the two. Below, fig. 3.9 and fig. 3.10, is given the training and test epoch loss and accuracy of models A and B using 10 epochs and a batch size of 128 units.

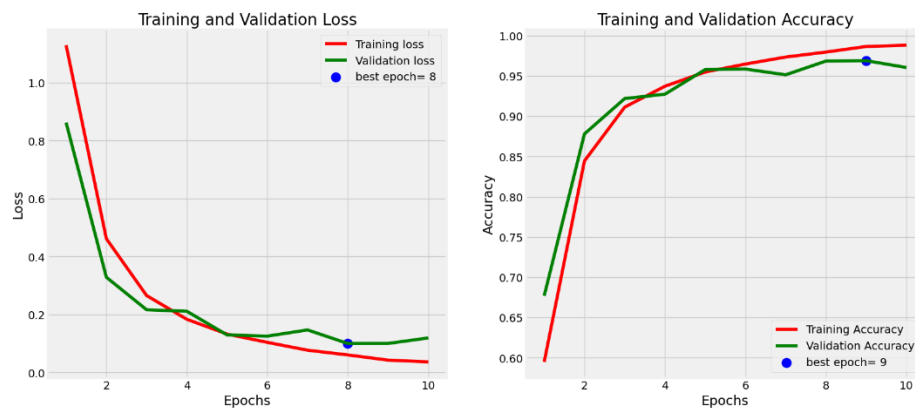


Fig. 3.9: Model A training and test epoch a) loss and b) accuracy respectively.

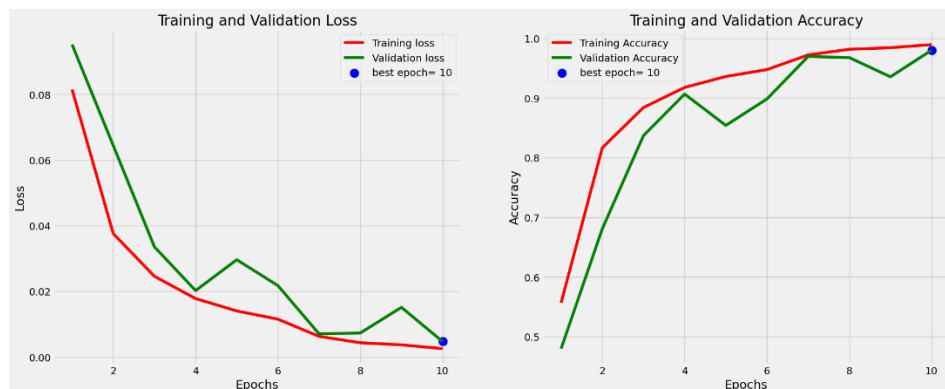


Fig. 3.10: Model A training and test epoch a) loss and b) accuracy respectively.

Both the models, model A and model B, have a similar architecture except a few features.

3.6.1 Summary of Model A

Model A is a Convolutional Neural Network (CNN) using the Adamax optimizer, specifically structured for image classification. The model leverages a series of convolutional layers, batch normalization, pooling, and dropout to extract features, prevent overfitting, and increase classification accuracy as shown in fig. 3.11.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	896
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
batch_normalization (BatchNormalization)	(None, 14, 14, 32)	128
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
conv2d_2 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 64)	256
conv2d_3 (Conv2D)	(None, 7, 7, 128)	73,856
conv2d_4 (Conv2D)	(None, 7, 7, 128)	147,584
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	0
batch_normalization_2 (BatchNormalization)	(None, 3, 3, 128)	512
conv2d_5 (Conv2D)	(None, 3, 3, 256)	295,168
conv2d_6 (Conv2D)	(None, 3, 3, 256)	590,080
max_pooling2d_3 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten (Flatten)	(None, 256)	0
dropout (Dropout)	(None, 256)	0
dense (Dense)	(None, 256)	65,792
batch_normalization_3 (BatchNormalization)	(None, 256)	1,024
dense_1 (Dense)	(None, 128)	32,896
batch_normalization_4 (BatchNormalization)	(None, 128)	512
dense_2 (Dense)	(None, 64)	8,256
batch_normalization_5 (BatchNormalization)	(None, 64)	256
dense_3 (Dense)	(None, 32)	2,080
batch_normalization_6 (BatchNormalization)	(None, 32)	128
classifier (Dense)	(None, 7)	231

Total params: 1,275,079 (4.86 MB)
Trainable params: 1,273,671 (4.86 MB)
Non-trainable params: 1,408 (5.50 KB)

Fig. 3.11: Model A summary

- The input layer takes in images of shape 28x28 pixels with RGB colour channels.

- Several Conv2D layers, with varying filter sizes (e.g., 32, 64, 128, and 256), use the ReLU activation function to highlight essential image features.
- he_normal initialization is used for these layers which helps to improve the performance and convergence speed by setting the initial weights based on the ReLU function.
- MaxPooling2D layers reduce spatial dimensions, preserving crucial features while making the model robust against variations in input.
- Batch Normalization layers are added to normalize and stabilize layer outputs, accelerating training and potentially increasing overall performance.
- A series of dense layers with ReLU activation provides a powerful classifier, with each dense layer connected to the subsequent one, reducing units from 256 to 32.
- Dropout is applied at a 0.2 rate to reduce overfitting by randomly deactivating neurons during training.

3.6.2 Summary of Model B

Model B adopts a deeper CNN architecture similar to Model A but integrates specific advanced features for refined learning. Model B uses Leaky ReLU activation and Mean Squared Error (MSE) loss, focusing on identifying subtle patterns in melanoma detection with added flexibility. Fig. 3.12 is explained as follows:

- The input layer processes images of 28x28 pixels with 3 colour channels.
- Convolutional layers here use LeakyReLU as an activation function with an alpha of 0.01, allowing a small gradient when the unit is not active, which helps avoid dead neurons.
- Layers are initialized with TruncatedNormal, which samples initial weights to avoid large gradients and improves stability.
- Pooling and batch normalization layers are applied ensuring that the model generalizes well while maintaining significant features.
- Dense layers allow the model to capture more complex relationships and apply LeakyReLU for activation.
- 20 percent neurons are randomly dropped to avoid overfitting.
- A softmax-activated dense layer categorizes images into the seven classes.

- The Mean Squared Error is employed as the loss function.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 28, 28, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 32)	0
batch_normalization_7 (BatchNormalization)	(None, 14, 14, 32)	128
conv2d_8 (Conv2D)	(None, 14, 14, 64)	18,496
conv2d_9 (Conv2D)	(None, 14, 14, 64)	36,928
max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 64)	0
batch_normalization_8 (BatchNormalization)	(None, 7, 7, 64)	256
conv2d_10 (Conv2D)	(None, 7, 7, 128)	73,856
conv2d_11 (Conv2D)	(None, 7, 7, 128)	147,584
max_pooling2d_6 (MaxPooling2D)	(None, 3, 3, 128)	0
batch_normalization_9 (BatchNormalization)	(None, 3, 3, 128)	512
conv2d_12 (Conv2D)	(None, 3, 3, 256)	295,168
conv2d_13 (Conv2D)	(None, 3, 3, 256)	590,080
max_pooling2d_7 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten_1 (Flatten)	(None, 256)	0
dropout_1 (Dropout)	(None, 256)	0
dense_4 (Dense)	(None, 256)	65,792
batch_normalization_10 (BatchNormalization)	(None, 256)	1,024
dense_5 (Dense)	(None, 128)	32,896
batch_normalization_11 (BatchNormalization)	(None, 128)	512
dense_6 (Dense)	(None, 64)	8,256
batch_normalization_12 (BatchNormalization)	(None, 64)	256
dense_7 (Dense)	(None, 32)	2,080
batch_normalization_13 (BatchNormalization)	(None, 32)	128
classifier (Dense)	(None, 7)	231

Total params: 1,275,079 (4.86 MB)
Trainable params: 1,273,671 (4.86 MB)
Non-trainable params: 1,408 (5.50 KB)

Fig. 3.12: Summary of model B

3.6.3 The Ensemble model

The strengths of model A and model B are strategically combined to obtain an enhanced accuracy. Individual strengths of the two models help in the creation of a robust model. Model A's speed and general pattern recognition is blended with the detailed analysis of model B to provide a system which is especially valuable in complex situations.

```

input_shape = (28, 28, 3) # Adjust to match your model's input shape
inputs = Input(shape=input_shape)

# Get predictions from both models
predA = modelA(inputs)
predB = modelB(inputs)

# Average the predictions
ensemble_output = Average()([predA, predB])

# Create the ensemble model
ensemble_model = Model(inputs=inputs, outputs=ensemble_output)

# Save the ensemble model as a SavedModel or HDF5 file
ensemble_model.save("ensemble_model_saved.keras")

```

Fig 3.13: Code snippet of ensemble model

Ensemble predictions are formed by merging the outputs of both model A predictions and model B predictions. Each prediction is calculated separately and the average of the two is used as ensemble prediction.

As given in fig. 3.13, the model is then saved as a Keras model.

Given below, fig. 3.14 represents the architectural structure of the ensemble model.

- The input layer receives data in the shape of 28x28 pixel images with RGB colour channels.
- 'None' denotes the batch size and therefore, allows the model to process variable number of input images at once.
- The two Sequential models represent the two models-model A and model B with an input shape of 28x28 pixels and RGB colour channels along with a variable batch size. The output is a probability score of each class prediction of the seven classes present in the dataset.
- The average layer then combines the outputs of the two sequential models and averages them.
- This leads to a more stable and reliable prediction.

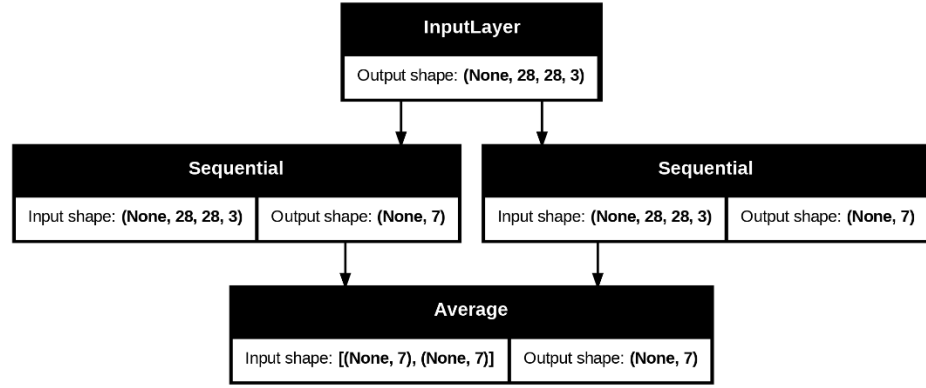


Fig. 3.14: Architecture of ensemble model

3.6.4 Comparison between model A and model B

Both Model A and Model B have an identical architecture with 26 layers each. However, they have differing loss functions, activation function and kernel initializers. The difference in the architecture is described as follows:

a) Loss Function: It measures the difference between the predicted output and the expected value and serves to the purpose of minimizing this difference.

Categorical Cross Entropy is used in model A. CCE is the measure of the difference between the probability distributions of the predicted class and the actual class labels as shown in eq. 3. It is a loss function where the outputs can belong to multiple classes and is especially helpful in cases involving classification. It calculates a loss value based on the confidence of the predictions. If the predicted probability for the correct class is high, the CCE loss is small, indicating a confident and correct prediction.

$$CCE = - \sum_{i=1}^c y_i \log(\hat{y}_i) \quad (3)$$

Mean Squared Error is also known as L2 loss. It is used in model B as the loss function. It is one of the most commonly used loss functions in machine learning and statistics, particularly for regression tasks. It calculates the square of the error difference between the predicted values and target values as shown in eq. 4. MSE quantifies how well a model's predictions align with the true values.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (4)$$

b) Activation function: They are functions used to transform an input signal to an output signal which is then passed as input to the next neural network layer. They help make the network dynamic and allows the extraction of complex features and represent non-linear relationships between the input and output.

Model A uses ReLU, Rectified Linear Unit, which is a mathematical function. It assigns zero to values less than zero and gives an output of x for an input value of x as given in eq. 5. A drawback of ReLU is the dying ReLU problem which is when a neuron always outputs zero for all inputs due to large negative values and thus, not contributing to any learning.

$$f(x) = \max(0, x) \quad (5)$$

Model B uses Leaky ReLU which is a mathematical function which reduces the possibility of having 0 as output. It is a modified version of ReLU that aims to solve the dying ReLU problem. It introduces a small non-zero gradient ' α ' when the input is negative as given in eq. 6.

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (6)$$

c) Weight initialization: Kernel weight initializers define how the weights in the convolutional layers are initialized before training begins. Proper initialization prevents issues like vanishing/exploding gradients and accelerates convergence.

Model A uses a normal initializer. It is also called “he_normal” and is designed specifically for layers using the ReLU activation function. The initializer assigns weights by sampling from a Gaussian distribution with a mean of zero.

Model B, on the other hand, uses a truncated normal initializer. It is often used with Leaky ReLU and other complex activations and samples weights from a normal distribution with a mean of zero just like he_normal. The difference arises when a value beyond two standard deviations is faced from the mean by discarding them. This prevents outlier values and reduces instability.

CHAPTER 4: RESULTS AND DISCUSSIONS

4.1 Model performance metrics

The metrics used to evaluate the machine learning models and their effectiveness are accuracy, precision, F1-score and confusion matrix. These metrics allow us to quantify the reliability and robustness of the models and further assess their suitability.

- **Accuracy:** Accuracy is the ratio of correctly classified instances to the total number of instances. It provides a general sense of how often the model predicts the correct class.
- **Precision:** Precision measures the proportion of true positive predictions out of all positive predictions made by the model.
- **F1-score:** The F1-score is the harmonic mean of precision and recall, providing a single metric that balances the trade-off between the two.
- **Confusion matrix:** A confusion matrix is a tabular representation of a model's predictions versus the actual labels.

4.2 Model A evaluation

Model A leverages a deep convolutional neural network architecture optimized for classifying melanoma. It employs multiple convolutional layers with ReLU activation and Normal initialization, ensuring efficient gradient propagation and fast convergence. MaxPooling layers reduce spatial dimensions, retaining essential features while minimizing computational load. Batch normalization is used extensively to stabilize learning, and dropout layers help prevent overfitting. The fully connected layers at the end incorporate L1L2 regularization to enhance generalization. The model concludes with a dense output layer using a softmax activation, classifying inputs into seven distinct categories.

4.2.1 Training results

Model A achieved commendable performance during training, indicating its ability to learn patterns effectively from the dataset.

- Accuracy: 0.9787
- Precision: 0.9796
- Recall: 0.9787
- F1-Score: 0.9787

4.2.2 Testing results

Model A achieved a high-test accuracy showcasing its effectiveness in melanoma detection. Key performance metrics on the test set include:

- Accuracy: 0.9652
- Precision: 0.9657
- Recall: 0.9652
- F1-Score: 0.9650

These metrics highlight the model's reliability in identifying melanoma cases with minimal false positives and negatives. Training and validation accuracy curves indicate steady learning without significant overfitting, demonstrating the robustness of architectural choices like dropout and regularization.

4.2.3 Benefits and drawbacks

Model A performs effective feature extraction and captures fine details in the dermoscopic images with the help of the multiple convolutional layers present in its structure.

The reduction in accuracy during testing indicates model A's struggle with handling variations in unseen data. Highly ambiguous or low-quality images can also lead to misclassifications.

4.3 Model B evaluation

Model B introduces slight architectural variations, utilizing Leaky ReLU activation and Truncated Normal initialization for kernel weights. The architecture closely resembles Model A, with convolutional layers followed by max pooling, batch normalization, and dropout. The Leaky ReLU activation mitigates issues of "dying neurons" or the "leaky ReLU problem" ensuring gradients flow through even when input value is less than zero. The final dense layers use similar regularization techniques, and the softmax output layer remains consistent with Model A for classification.

4.3.1 Training results

Model B showed superior training performance, reflecting its optimized architecture with near-perfect classification on the training dataset with minimal missed detections.

- Accuracy: 0.9908
- Precision: 0.9909
- Recall: 0.9908
- F1-Score: 0.9908

4.3.2 Testing results

On the test dataset, Model B delivered impressive results as given below:

- Accuracy: 0.9816 indicates exceptional adaptability to new data.
- Precision: 0.9815 shows minimal false positives in melanoma detection.
- Recall: 0.9816 reflects the model's reliability in capturing true positives.
- F1-Score: 0.9814 emphasizes consistent performance across metrics.

4.3.3 Benefits and drawbacks

Model B provides improved generalization capabilities by addressing the "dying neuron" problem by using leaky ReLU. This enables the model to extract features effectively, even from noisy data. Stable weight updates throughout the learning process through truncated normal initialization smoothen the training process. Both the

training and testing metrics indicate that model B adapts exceptionally well to varying data distributions.

Model B, despite its performance, has a few drawbacks. The minimal gap between training and testing metrics suggests potential overfitting in certain cases, requiring regularization enhancements. It also requires careful tuning of hyperparameters.

4.4 Final Results

In the results of our melanoma detection project, we achieved commendable performance from the integrated machine learning models. The first model is a convolutional neural network (CNN), designed to extract intricate features from images of skin lesions, using Adamax optimizer. It performs quickly and effectively. It attained a test accuracy of 0.9652 and a precision of 0.9657 as given in fig 4.1.

```
ModelA Accuracy: 0.9652
ModelA Precision: 0.9657
ModelA Recall: 0.9652
ModelA F1-Score: 0.9650
```

Fig. 4.1: Test Accuracy of Model A

The second model adopts a deeper CNN architecture similar to model A but, integrated specific advanced features for refined learning. It is highly robust, but its complexity might increase training time and computational requirements. It received a test accuracy of 0.9816 as given in fig. 4.2.

```
294/294 ————— 16s 54ms/step
ModelB Accuracy: 0.9816
ModelB Precision: 0.9815
ModelB Recall: 0.9816
ModelB F1-Score: 0.9814
```

Fig. 4.2: Test Accuracy of model B

The ensemble model combines the strengths of model A and model B, and aims to balance model A's efficiency with model B's ability to capture complex details. The

average of the two models' predictions makes the model more robust and reduces the risk of overfitting to a specific feature set as seen in fig. 4.3.

```
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
```

```
Accuracy: 0.9850
Precision: 0.9851
Recall: 0.9850
F1-Score: 0.9848
```

Fig. 4.3: Test Accuracy of ensemble model

The confusion matrix, fig. 4.4, indicated high accuracy of the ensemble model and highlights that it made 1359 correct predictions of elements of class 6 i.e. melanoma.

The correlation matrix visualizes the relationship between all the possible pairs between the different predictions of the two models. The negative values of the non-diagonal elements in fig. 4.5 represent weak negative correlations between the different predictions.

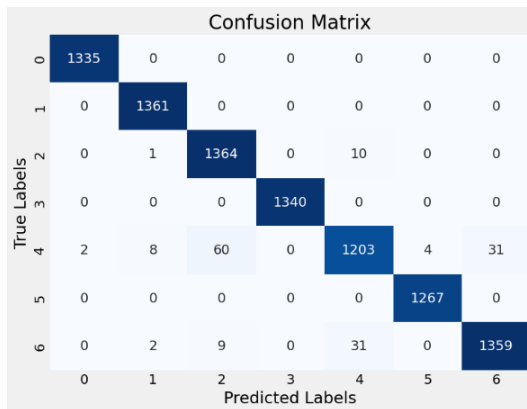


Fig. 4.4: Confusion matrix of ensemble model

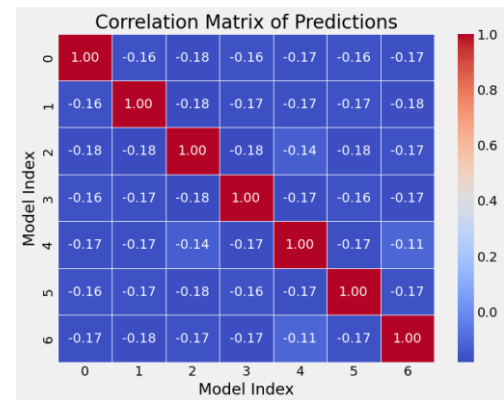


Fig. 4.5: Correlation matrix

Model A and model B are CNN models with custom activation function and dense layers and uses convolutional and pooling layers for feature extraction and batch normalization to stabilize the learning. Model A uses MSE as loss function and ReLU as activation function to achieve an accuracy of 97.8. Model B on the other hand uses CCE as activation function and Leaky ReLU as activation function to achieve an

accuracy of 98.16. It is found that the aggregation of the two models provide the best results with the accuracy of 98.5 as seen in table 1.

S.no.	Models	Training Accuracy	Testing accuracy
1	Model A	97.8	96.52
2	Model B	99.08	98.16
3	Ensemble model of model A and model B	99.5	98.5

Table 1: Model accuracy comparison

4.5 Future Scope

This project demonstrates the transformative potential of machine learning in the field of dermatology, specifically in the detection and diagnosis of melanoma. By harnessing advanced CNNs and transfer learning, our system exemplifies how deep learning can amplify clinical workflows, providing valuable diagnostic insights that aid healthcare practitioners in assessing and monitoring skin health. This research underscores the role of cutting-edge technology in addressing pressing healthcare needs and highlights the potential of AI-powered models to support individuals and professionals in early melanoma diagnosis and proactive skin health management.

- This model can be integrated with a user-friendly mobile application for real-time diagnostics and monitoring of skin lesions. This will encourage users to keep check at any irregularities and determine when it is time to contact a doctor.
- It can be further be expanded to detect other skin diseases, therefore increasing its diagnostic utility.
- Architectural adjustments can be made by introducing attention layers, transformer models etcetera for more precise feature extraction and hence, an improved model overall.

- Implementation of continuous learning pipelines can be introduced to adapt to new and diverse data.
- Collaborating with medical institutions will allow access to a wider set of records making it possible to validate and refine the current model using real-world clinical data.
- This further gives room to the possibility of combining image data with patient history for a more comprehensive analysis.

CHAPTER 5: CONCLUSION

In conclusion, our final report highlights the progress and accomplishments of the melanoma detection project, marking a successful completion of all project objectives. An advanced deep learning model was built successfully for effective early melanoma detection. This model demonstrates both the depth of our technical approach and the potential impact on the field of dermatology.

The detailed architectural summaries of both Model A and Model B give an outline of their neural network structures, facilitating the extraction of features necessary for accurate classification and prediction. The convolutional layers capture the spatial hierarchies, while batch normalization ensures stable and efficient training by maintaining mean and standard deviation. Dropout layers resolve the problem of overfitting, enhancing the models' generalization capabilities. The dense classifier processes the acquired features to categorize skin lesions into predicting melanoma and non-melanoma classes, leading to a comprehensive diagnostic structure. These two models are then combined together by averaging the separate outputs of each to result in prediction of the ensemble. Through the acquired TFlite model, predictions can be made with up to 0.9850 accuracy.

This project exemplifies the high potential of merging deep learning with healthcare. The ability of our models to identify early-stage melanoma not only holds promise for early detection but, also represents a step forward in accessible, proactive healthcare solutions.

As we conclude, this project stands as a testament to the significant advancements that artificial intelligence can bring to the field of medical diagnostics, offering a promising avenue towards more accessible and effective healthcare.

REFERENCES

1. https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript<https://developer.mozilla.org/en-US/docs/Web/CSS>
2. <https://my.clevelandclinic.org/health/diseases/14391-melanoma>
3. <https://www.mayoclinic.org/diseases-conditions/melanoma/symptoms-causes/syc-20374884>
4. <https://www.ibm.com/topics/deep-learning>
5. <https://www.geeksforgeeks.org/introduction-deep-learning/>
6. A. A. Adegun and S. Viriri, "Deep Learning-Based System for Automatic Melanoma Detection," in IEEE Access, vol. 8, pp. 7160-7172, 2020, doi: 10.1109/ACCESS.2019.2962812, <https://ieeexplore.ieee.org/abstract/document/8945133/references#references>
7. Viknesh, C. K., Kumar, P. N., Seetharaman, R., & Anitha, D. (2023). Detection and Classification of Melanoma Skin Cancer Using Image Processing Technique. *Diagnostics (Basel, Switzerland)*, 13(21), 3313. <https://doi.org/10.3390/diagnostics13213313>
8. Moturi, D., Surapaneni, R.K. & Avanigadda, V.S.G. Developing an efficient method for melanoma detection using CNN techniques. *J Egypt Natl Canc Inst* **36**, 6 (2024). <https://doi.org/10.1186/s43046-024-00210-w>
9. Balazs Harangi, Skin lesion classification with ensembles of deep convolutional neural networks, *Journal of Biomedical Informatics*, Volume 86, 2018, Pages 25-32, ISSN 1532-0464, <https://doi.org/10.1016/j.jbi.2018.08.006>.
10. C. P., M. ., P. Jo, J. ., & V., V. . (2023). Computer-Aided Detection of Skin Cancer Detection from Lesion Images Via Deep Learning Techniques: 3d CNN Integrated Inception V3 Networks. *International Journal of Intelligent Systems and*

- Applications in Engineering*, 11(9s), 550–562. Retrieved from <https://ijisae.org/index.php/IJISAE/article/view/3191>
11. Kumar Lilhore, U., Simaiya, S., Sharma, Y.K. *et al.* A precise model for skin cancer diagnosis using hybrid U-Net and improved MobileNet-V3 with hyperparameters optimization. *Sci Rep* **14**, 4299 (2024). <https://doi.org/10.1038/s41598-024-54212-8>
 12. Li Y, Shen L. Skin Lesion Analysis towards Melanoma Detection Using Deep Learning Network. *Sensors*. 2018; 18(2):556. <https://doi.org/10.3390/s18020556>
 13. Jinen Daghrir, Lotfi Tlig, Moez Bouchouicha, Mounir Sayadi. Melanoma skin cancer detection using deep learning and classical machine learning techniques: A hybrid approach. International Conference on Advanced Technologies for Signal and Image Processing, Sep 2020, Sfax, Tunisia. ff10.1109/ATSIP49331.2020.9231544ff. fffhal-03172718 <https://hal.science/hal-03172718/file/PID6365049.pdf>
 14. Banerjee, S., Singh, S. K., Chakraborty, A., Das, A., & Bag, R. (2020). Melanoma diagnosis using deep learning and fuzzy logic. *Diagnostics*, 10(8), 577. <https://www.mdpi.com/2075-4418/10/8/577>
 15. Alwakid, G., Gouda, W., Humayun, M., & Sama, N. U. (2022, December). Melanoma detection using deep learning-based classifications. In *Healthcare* (Vol. 10, No. 12, p. 2481). MDPI. <https://doi.org/10.3390/healthcare10122481>
 16. Ansari, U. B., & Sarode, T. (2017). Skin cancer detection using image processing. *Int. Res. J. Eng. Technol*, 4(4), 2875-2881. <https://www.academia.edu/download/53555135/IRJET-V4I4702.pdf>
 17. Kousis, I., Perikos, I., Hatzilygeroudis, I., & Virvou, M. (2022). Deep learning methods for accurate skin cancer recognition and mobile application. *Electronics*, 11(9), 1294. <https://doi.org/10.3390/electronics11091294>
 18. Dascalu, A., & David, E. O. (2019). Skin cancer detection by deep learning and sound analysis algorithms: A prospective clinical study of an elementary dermoscope. *EBioMedicine*, 43, 107-113.
 19. Jojoa Acosta, M. F., Caballero Tovar, L. Y., Garcia-Zapirain, M. B., & Percybrooks, W. S. (2021). Melanoma diagnosis using deep learning techniques on dermatoscopic images. *BMC Medical Imaging*, 21, 1-11. <https://link.springer.com/content/pdf/10.1186/s12880-020-00534-8.pdf>

20. Nawaz, M., Mehmood, Z., Nazir, T., Naqvi, R. A., Rehman, A., Iqbal, M., & Saba, T. (2022). Skin cancer detection from dermoscopic images using deep learning and fuzzy k-means clustering. *Microscopy research and technique*, 85(1), 339-351.
<https://doi.org/10.1002/jemt.23908>
21. <https://datasetninja.com/skin-cancer-ham10000#introduction>
22. <https://www.geeksforgeeks.org/introduction-convolution-neural-network/>
23. <https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>
24. https://www.w3schools.com/python/python_intro.asp
25. <https://www.w3schools.com/python/numpy/default.asp>
26. <https://www.tensorflow.org/>
27. https://keras.io/getting_started/
28. https://www.w3schools.com/python/pandas/pandas_intro.asp
29. O'Shea, Keiron & Nash, Ryan. (2015). An Introduction to Convolutional Neural Networks. ArXiv e-prints.
https://www.researchgate.net/publication/285164623_An_Introduction_to_Convolutional_Neural_Networks
30. Terven, Juan & Cordova-Esparza, Diana-Margarita & Ramirez-Pedraza, Alfonso & Chávez Urbiola, Edgar. (2023). Loss Functions and Metrics in Deep Learning. A Review. 10.48550/arXiv.2307.02694.
31. Sharma, Sagar, Simone Sharma, and Anidhya Athaiya. "Activation functions in neural networks." *Towards Data Sci* 6.12 (2017): 310-316.
<https://www.ijeast.com/papers/310-316,Tesma412,IJEAST.pdf>

32. <https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>
33. <https://www.analyticsvidhya.com/blog/2018/04/fundamentals-deep-learning-regularization-techniques/>
34. <https://www.analyticsvidhya.com/blog/2022/08/dropout-regularization-in-deep-learning/>